

In RAG We Trust? Ensemble Models for Retrieval Augmented Generation

Christian R. Cuenca
Dipartimento di Informatica
Università degli Studi di Milano
Milan, Italy
christian.cuenca@studenti.unimi.it

Abstract—Retrieval-Augmented Generation (RAG) systems are increasingly used alongside Large Language Models (LLMs), largely because they can ground outputs in domain specific knowledge. This is particularly relevant in enterprise settings such as compliance, governance, security policies, and internal procedures, where answers should be traceable to authoritative sources and mistakes may have operational or regulatory consequences. However, RAG pipelines remain vulnerable to weak retrieval and to retrieval poisoning, where misleading evidence is introduced into the knowledge base. In this work we present a RAG pipeline in which multiple role-specialized LLMs independently produce evidence-grounded answers from the same retrieved context, and a final Judge model consolidates these candidates into a single response.

Index Terms—RAG, LLM Ensembling, Poisoning, Prompt Engineering

I. INTRODUCTION

Managing information in the era of rapidly emerging technologies has become increasingly important, especially when inputs and outputs are produced by systems such as LLMs. This need is even more pressing given the spread of fake news and disinformation on social media. In sensitive domains, misinformation can have tangible consequences, including public unrest [1], and the risk is amplified when narratives involve elections or pandemics [2], [3].

Automated verification methods can help mitigate these effects. Several approaches have been proposed in the literature. For instance, Singhal et al. [4] describe an evidence-backed fact verification pipeline that classifies claims into categories such as Supported, Refuted, Conflicting Evidence/Cherry-picking, or Not Enough Evidence.

A related line of work aims to improve the reliability of LLM outputs by grounding them in external sources: Lewis et al. [5] introduced Retrieval-Augmented Generation (RAG), where a generative model retrieves relevant documents from an index (e.g., Wikipedia) before or during answer generation. This design addresses two practical issues: (i) model knowledge is largely encoded in parameters and may become outdated unless the model is retrained, and (ii) answers can be supported with explicit provenance.

Today, RAG systems are widely adopted to enhance LLM performance and reduce hallucinations, particularly in domain-specific settings. A concrete example is the legal domain: Buffa et al. [6] propose JusBuild, a RAG-based architecture

intended to support legal practitioners in working with legal documents.

Despite these advantages, RAG pipelines are not error-free and may still produce inaccurate or misleading outputs. Zhou et al. [7] propose a benchmark framework to evaluate trustworthiness across six dimensions:

- i. Factuality, accuracy and truthfulness of the information generated
- ii. Robustness, system’s reliability in resisting errors and external threats
- iii. Fairness, minimize bias and ensure equitable treatment of all users
- iv. Transparency, making the processes and decisions of the system clear and understandable to users
- v. Accountability, mechanisms that hold the system responsible for its actions and outputs
- vi. Privacy, ensures the protection of personal data and user privacy

In this work we present a RAG system where multiple role-specialized LLMs deliberate within a Council to improve response quality (Figure 1), inspired by Karpathy’s LLM Council concept [8]. We situate this work in a corporate domain-specific setting focused on cybersecurity governance, and we evaluate the system behavior under controlled conditions where the underlying vector database is subject to retrieval poisoning.

II. MATERIALS AND METHODS

A. Dataset

The dataset used in this study is synthetic, i.e., it was artificially generated using a commercial LLM, specifically OpenAI’s ChatGPT 5.2. The reference corpus consists of 10 documents. For ease of handling within the preprocessing pipeline, all documents are stored in plain-text format (.txt). In a real-world deployment, the same material would typically be distributed as PDFs or other heterogeneous formats, requiring more advanced preprocessing (e.g., layout parsing and robust text extraction).

The documents are designed to reflect enterprise management documentation in a corporate setting, with a focus on the banking domain. They simulate internal policies and operational procedures relevant to an ICT organizational unit,

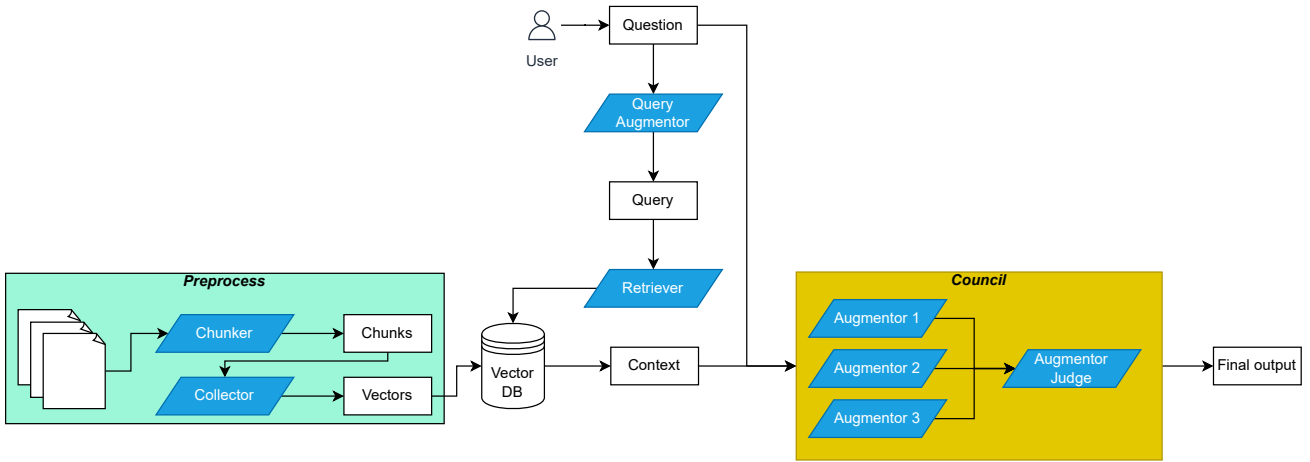


Fig. 1. Overview of the proposed RAG-based pipeline. During preprocessing, documents are segmented into chunks by the Chunker. The Collector encodes each chunk into a dense vector and indexes it in the vector database. At inference time, a user question is first reformulated by a query-enhancement model, which generates one or more retrieval queries. The Retriever performs top- k similarity search over the vector index and aggregates the retrieved passages into a supporting context. The original question and the retrieved context are then provided to the Council, i.e., an ensemble of role-specialized generative models producing candidate answers. Finally, a Judge model validates these candidates and outputs the final response.

including security and governance oriented guidance. Such documents play a central role in organizational decision-making, particularly for information security and compliance-related workflows.

In terms of size, documents contain on average 984.3 words. The longest document includes 1865 words, while the shortest includes 823 words. Structurally, each document is organized into numbered sections with headings, and some documents include additional subsections to mirror common corporate documentation conventions.

B. Document preprocessing

The dataset is processed by a `Chunker` module whose objective is to segment each `.txt` document into section-level chunks based on numbered headings (e.g., “1. Introduction”). To improve interpretability and facilitate manual inspection, each extracted section is also exported as an individual `.txt` file while preserving associated metadata. Specifically, each chunk contains all lines belonging to a given section, including the section number and title, and is stored without semantic rewriting, normalization, nor content cleaning.

The original ordering of sections is retained through a progressive indexing scheme (e.g., `chunk_001`, `chunk_002`). Each chunk is additionally linked to its source document via metadata that records the original filename, enabling traceability from retrieved passages back to the originating document. The total number of chunks is 204.

The `Collector` module reads each generated chunk, extracting both its semantic content and the associated metadata, and then computes dense sentence-level embeddings using the lightweight Sentence-Transformers model `all-MiniLM-L6-v2`. The resulting vectors, together with the corresponding metadata, are stored in a persistent ChromaDB collection, enabling similarity-based retrieval.

Embeddings are computed only from the section body text, excluding the section number and title. This design choice aims to improve the semantic quality of the vector representations, and enhance query generalization during retrieval.

C. Inference flow

At inference time, the user provides a natural-language question to the system. The question is first handled by the `Query Augmentor`, an LLM-based module that reformulates the original input into one or more enhanced retrieval queries. The goal is to improve retrieval effectiveness by emphasizing semantically informative terms and reducing the impact of vague or low-signal tokens that could otherwise degrade similarity matching.

The `Retriever` then encodes the enhanced queries into dense vectors using the same embedding model adopted during indexing and performs semantic vector search over the ChromaDB collection. Similarity is computed using cosine similarity, and nearest-neighbor retrieval is implemented via HNSW (Hierarchical Navigable Small World), which enables efficient approximate search in high-dimensional embedding spaces. The retriever returns the top- k most similar vectors along with their associated metadata, which are subsequently used to construct the evidence context for downstream generation.

The original user question, together with the retrieved context from the vector database, is provided to a Council of `Augmentor` models. Each `Augmentor` is an LLM instantiated with a role-specific system prompt designed to elicit complementary perspectives over the same evidence. For instance, prompts include roles such as *a conservative banking compliance expert*, *a pragmatic ICT operations engineer*, and *a critical security auditor*.

Augmentation is implemented through a structured prompt template shared across all Augmentor models as follows:

You are {role prompt}
 Answer using ONLY the context below.
 Context: {context}
 Question: {question}

This formulation enforces the evidence base by discouraging the use of outside knowledge and ensures that differences in candidate responses arise primarily from the specific definition of the role rather than from access to different sources.

Finally, all candidate answers produced within the Council are reviewed by a dedicated Augmentor Judge, whose objective is to consolidate the Council’s outputs into a single response. The Judge analyzes (i) the original user question, (ii) the retrieved evidence context, and (iii) the set of candidate answers, with the explicit goal of resolving inconsistencies and selecting the most defensible interpretation supported by the evidence. The Judge is prompted as follows:

You are a senior banking ICT and security reviewer.
 Question: {question}
 Context: {context}
 Answers from multiple experts: {multi-answers}
 Task:
 - Compare the answers
 - Resolve contradictions
 - Produce a single, accurate, well-justified final answer
 - Use ONLY the provided context

After this step, the court’s output is returned as the system’s final answer. All experiments were conducted using OpenAI’s gpt-4.1-nano model.

III. RESULTS

To conduct the experiments, we constructed a test set of 50 questions, each annotated with the expected source document and the expected document section. In addition, 5 out-of-corpus questions were intentionally included to evaluate system behavior when the answer is not present in the vector database.

Following the trustworthiness framework proposed by Zhou et al. [7], this section focuses on the Factuality dimension, with particular attention to how the system combines its parametric knowledge (internal to the model) with retrieved evidence (external context). We also run a dedicated experiment under retrieval poisoning, assessing performance when the vector database contains misleading or adversarially injected content.

A. Experiment-Factuality

In this experiment, we investigate scenarios where the model’s internal (parametric) knowledge conflicts with the external evidence retrieved from the document corpus. To support this analysis, we modify the prompting strategy for the Council models as follows:

You are {role prompt}
 Answer using also the context below. Be concise.

Context: {context}
 Question: {question}
 Output in this format:
 Answer: <your brief answer>
 Used Model’s Internal Knowledge: <YES if you used internal knowledge rather than the provided but yet non relevant source, else NO>

The Judge prompt is adapted accordingly:

You are a senior banking ICT and security reviewer
 Question: {question}
 Context: {context}
 Answers from multiple experts: {multi-answers}
 Task:
 - Compare the answers
 - Resolve contradictions
 - Produce a single, accurate, well-justified final answer
 - Be concise. Say clearly what is the final answer providing only the name of the source file
 - Provide the answer with the following format:
 Final Answer: <your BRIEF final answer here>
 Used Model’s Internal Knowledge: <YES if model has used internal knowledge rather than the provided yet non relevant source, else NO>

For retrieval we set top- k to $k = 1$, i.e., the Retriever returns only the single most relevant passage. This choice is deliberate: by restricting the context to one chunk, we increase the likelihood that the system is exposed to incomplete or partially relevant evidence, allowing us to observe how it behaves when retrieval support is weak, e.g. whether it defaults to internal knowledge rather than adhering to the provided context.

The results were carefully reviewed by a domain expert. We evaluated both (i) the model-generated answers and (ii) the evidence retrieved by the Retriever for each query. For retrieval assessment, we annotated the retrieved document as ‘YES’ when it matched the expected source document, ‘YES BUT’ when it was not the expected document but still partially relevant or contained overlapping evidence, and ‘NO’ when it was not coherent with the question. The metric used is Exact Match@1 (EM@1) defined as follows:

$$EM@1 = \frac{|YES|}{|Questions|} \quad (1)$$

We also use Acceptable@1 (Acc@1), Irrelevant@1 (Err@1) and Off Target But Relevant (OTBR) defined as follows:

$$Acc@1 = \frac{(|YES| + |YES BUT|)}{|Questions|} \quad (2)$$

$$Err@1 = \frac{|NO|}{|Questions|} \quad (3)$$

$$OTBR = \frac{|YES BUT|}{|Questions|} \quad (4)$$

Under these metrics, the retriever achieves $EM@1 = 0.38$, $Acc@1 = 0.68$, $Err@1 = 0.32$, and $OTBR = 0.30$.

In 8 cases, the system explicitly stated that it relied on its parametric knowledge to answer. Regarding the 5 out-of-corpus questions, the system generally responded appropriately by indicating that the required information was not present in the provided documents. In one of these cases, the model still produced a correct answer using internal knowledge, i.e., by leveraging common knowledge rather than the retrieved evidence.

B. Experiment-Poisoning

To evaluate the system under adversarial conditions, i.e. retrieval poisoning, we again follow the approach of Zhou et al. [7]. For each question in the test set, we replaced the retrieved evidence with passages that are topically relevant but factually incorrect. We considered multiple poisoning strategies:

- Numerical flips (e.g., SLA timelines, retention periods, thresholds; “within X days $\rightarrow Y$ days”)
- Role swaps (e.g., inverting responsibilities across HR, IAM, and the CISO)
- Control inversion (e.g., “MUST encrypt” $\rightarrow$ “encryption optional”)
- Exception injection (e.g., “for urgent cases, approvals can be bypassed”)

Poisoning intensity varied across questions: some items were left unchanged, others were partially or fully poisoned, and a subset was poisoned by introducing explicit contradictions within the retrieved context. Numerically, out of 50 questions, 43 were poisoned in a subtle manner (where 8 left unchanged), 7 were poisoned in an obvious manner.

The prompting strategy for the Council models is modified as follows:

```
You are {role prompt}
Answer using also the context below. Be concise.
Context: {context}
Question: {question}
Output in this format:
***Answer***: <your brief answer>
***Used Model's Internal Knowledge***: <YES if you used
internal knowledge rather than the provided but yet non relevant
source, else NO>
***Poisoned***: <FULL if you think (base on your expertise
in the field) that the provided source could be poisoned,
PARTIALLY if you think that the source could be partially
poisoned, NO if you think is normal>
```

The Judge prompt is adapted accordingly:

```
You are a senior banking ICT and security reviewer
Question: {question}
Context: {context}
Answers from multiple experts:{multi-answers}
Task:
- Compare the answers
- Resolve contradictions
- Produce a single, accurate, well-justified final answer
- Be concise. Say clearly what is the final answer providing
only the name of the source file
- Provide the answer with the following format:
```

```
***Final Answer***: <your BRIEF final answer here>
***Used Model's Internal Knowledge***: <YES if model
has used internal knowledge rather than the provided yet non
relevant source, else NO>
***Poisoned***: <FULL if (base on experts answer) the pro-
vided source could be poisoned, PARTIALLY if the source
could be partially poisoned, NO if normal>
```

To assess performance in this adversarial setting, we measure how often the model explicitly flags the provided evidence as fully or partially poisoned. Across the 50 questions, the system identified 1 case as fully poisoned and 1 case as partially poisoned.

IV. DISCUSSION AND CONCLUSION

In this work, we assess the factuality dimension of RAG trustworthiness following the methodology proposed by Zhou et al. [7]. To stress-test factuality, we deliberately constrained retrieval by setting top- k to $k = 1$. This choice is motivated by the fact that, with larger k , the system would eventually retrieve at least one truly relevant passage and thus answer correctly more often, masking failure modes that emerge under weak or noisy evidence.

The retriever’s performance (e.g., EM@1) is also influenced by our preprocessing design: chunks are embedded using only the section body text, excluding section titles. This yields semantically cleaner representations, but it also introduces cases where the most similar chunk is not the expected one, despite being partially informative. For example, for the question “Describe the ICT risk governance structure and responsibilities”, the expected evidence is the roles and responsibilities section of *ICT_Risk_Management_Policy.txt*, but instead the retriever returns the introduction section, which still contains generic but relevant governance information.

These experiments further indicate that when retrieval is not sufficiently aligned with the question, the system may fall back on parametric knowledge despite being instructed to rely only on the provided context. Notably, reliance on parametric knowledge is not uniform across the Council: in some instances only a subset of Augmentors deviates from the evidence constraint. In the final output, the Judge reports parametric knowledge usage only when it reflects the majority of the Council (or the Judge’s own assessment)

For the intentionally out-of-corpus questions, the system generally behaved appropriately: it either stated that the answer could not be found in the provided documents, or it responded using internal knowledge. A representative example is “Which exact ISO 27001 controls (Annex A) are adopted as mandatory, with control IDs?”, where the model produced a correct answer based on its background knowledge of ISO 27001.

Regarding retrieval poisoning, within the poisoned subset (41 questions: 35 subtle and 6 obvious; the remaining items were left unchanged), the system flagged poisoning only twice: one case as fully poisoned (in obvious manner) and one as partially poisoned (in subtle manner).

Interestingly, when examining the Council outputs (excluding the Judge), we observed 4 instances where at least one

Augmentor raised a poisoning concern; however, these signals were not retained in the final adjudication, suggesting that the Judge step can suppress minority warnings even under the current prompting strategy.

We also observed that poisoning effectiveness depends on question–poison alignment. For instance, a role-swap injection (e.g., incorrectly assigning backup responsibilities to HR) has limited impact if the question targets implementation standards rather than accountability, such as “*What implementation standards are required for backups?*”.

Two practical mitigation directions emerge. First, increasing top- k can help “dilute” compromised evidence by providing additional passages that may counterbalance poisoned ones. Second, the prompting and decision rule of the Council/Judge could be tuned to adopt a more conservative aggregation strategy (e.g., treating any poisoning flag raised by a Council member as a positive signal) an approach often favored in security settings where false positives can be preferable to false negatives.

As future work, it would be valuable to study vector-level poisoning more directly, quantifying how often poisoned passages are retrieved and under which embedding/index configurations this becomes more likely.

Overall, the results support the practical value of RAG in enterprise contexts. In document-centric workflows, such as gap analysis and audit, RAG can simplify evidence access and synthesis, and an ensemble-based design can further improve reliability.

REFERENCES

- [1] M. Lindsay and C. Grewar, “Social media misinformation ‘fanned riot flames’,” BBC News, 2024, [Online]. Available: <https://www.bbc.com/news/articles/c70jz2r4lp0o>. Accessed: 2026-02-16.
- [2] A. Bovet and H. A. Makse, “Influence of fake news in twitter during the 2016 us presidential election,” *Nature Communications*, vol. 10, no. 1, Jan. 2019. [Online]. Available: <http://dx.doi.org/10.1038/s41467-018-07761-2>
- [3] J. Bae, D. Gandhi, J. Kothari, S. Shankar, J. Bae, P. Patwa, R. Sukumaran, A. Chharia, S. Adhikesaven, S. Rathod, I. Nandutu, S. TV, V. Yu, K. Misra, S. Murali, A. Saxena, K. Jakimowicz, V. Sharma, R. Iyer, A. Mehra, A. Radunsky, P. Katiyar, A. James, J. Dalal, S. Anand, S. Advani, J. Dhaliwal, and R. Raskar, “Challenges of equitable vaccine distribution in the covid-19 pandemic,” 2022. [Online]. Available: <https://arxiv.org/abs/2012.12263>
- [4] R. Singhal, P. Patwa, P. Patwa, A. Chadha, and A. Das, “Evidence-backed fact checking using rag and few-shot in-context learning with llms,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.12060>
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” 2021. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [6] M. Buffa, A. Ferrara, S. Picascia, D. Riva, and S. Castano, “Enhancing legal document building with retrieval-augmented generation,” *Computer Law & Security Review*, vol. 59, p. 106229, 2025.
- [7] Y. Zhou, Y. Liu, X. Li, J. Jin, H. Qian, Z. Liu, C. Li, Z. Dou, T.-Y. Ho, and P. S. Yu, “Trustworthiness in retrieval-augmented generation systems: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.10102>
- [8] A. Karpathy, “llm-council: Llm council works together to answer your hardest questions,” GitHub repository, Nov. 2025, [Online]. Available: <https://github.com/karpathy/llm-council>.